



Analytics Solutions Bootcamp: #03

Fundamentals of Data Engineering

A primer on designing and building robust data systems

Hello, everyone! Welcome to the third module of the Analytics Solutions Bootcamp. In this module, you will learn about the fundamentals of data engineering. We will talk about the data engineering life cycle, how we build data pipelines from scratch, how we model data, and how we serve data.

Topics to be covered



- What is data engineering?
- Data engineering lifecycle
 - Generation
 - Storage
 - Ingestion
 - Transformation and modeling
 - Serving
- Introduction to modern data stack

What is data engineering?

Definition and skills

Definition of and relevant skills related to data engineering

What is data engineering? Definition, history, skills

Data engineering defined

“Data engineering is the development, implementation, and maintenance of systems and processes that take in raw data and produce high-quality, consistent information that supports downstream use cases, such as analysis and machine learning.”

August 28, 2026

Analytics Solutions Bootcamp

4

Include “design”

What is data engineering? Definition, history, skills

Brief history of data engineering

- 1988 an IBM Systems Journal article mentioned “business data warehouse”
- 1992 Bill Inmon published “Building the Data Warehouse”
- 1996 Ralph Kimball published “The Data Warehouse Toolkit”
- Mid 1990s – dot-com bubble (dot-com boom)
- Early 2000s – big data era begun
- 2003 – Google published a paper on Google File System
- 2004 – a paper on MapReduce was published

What is data engineering? Definition, history, skills

Brief history of data engineering

- 2006 – Yahoo engineers open-sourced Apache Hadoop
- 2000s–2010s other big data engineering tools were introduced
- 2022 – inception of AI

What is data engineering? Definition, history, skills

Data engineering skills and activities

Data engineers are generally focused on the following metrics.

1. Cost
2. Agility
3. Scalability
4. Simplicity
5. Reuse
6. Interoperability

What is data engineering? Definition, history, skills

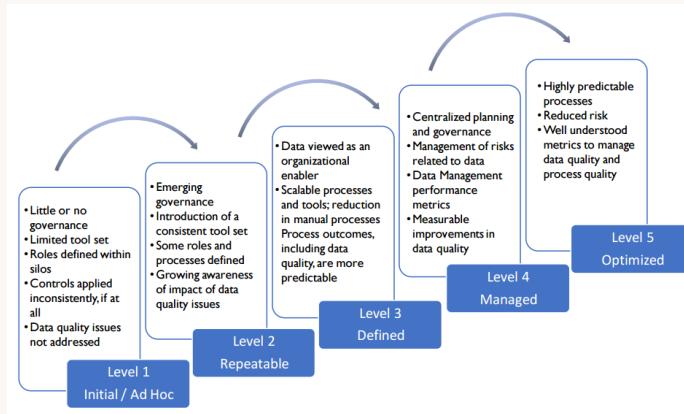
Data maturity of data engineers

Data maturity is the progression toward higher data utilization, capabilities, and integration across the organization.

1. Starting with data
2. Scaling with data
3. Leading with data

What is data engineering? Definition, history, skills

Data maturity model



August 28, 2026

Analytics Solutions Bootcamp

9

What is data engineering? Definition, history, skills

Background & skills of data engineers

Data engineers have two main responsibilities

- Business responsibilities
 1. Communicate with non-technical and technical people
 2. Understand how to scope and gather business requirements
 3. Understand Agile, DevOps, and DataOps
 4. Control costs
 5. Learn continuously

What is data engineering? Definition, history, skills

Background & skills of data engineers

Data engineers have two main responsibilities

- Technical responsibilities
 1. Data engineering lifecycle stages (generation to serving)
 2. Data engineering lifecycle undercurrents (security, data management, DataOps, data architecture, orchestration, software engineering)
 3. Tools: SQL, Python, JVM languages (Java, Scala), bash

Data engineering lifecycle

Generation, storage, serving

Understand the data engineering lifecycle and its undercurrents

Data engineering lifecycle

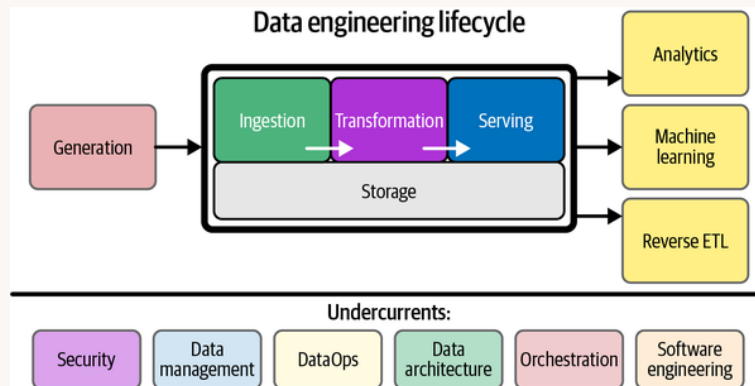
Data engineering lifecycle definition

The data engineering lifecycle comprises stages that turn raw data ingredients into a useful end-product, ready for consumption by analysts, data scientists, ML engineers, and others.

Data engineering lifecycle

Data engineering lifecycle

>>



August 28, 2026

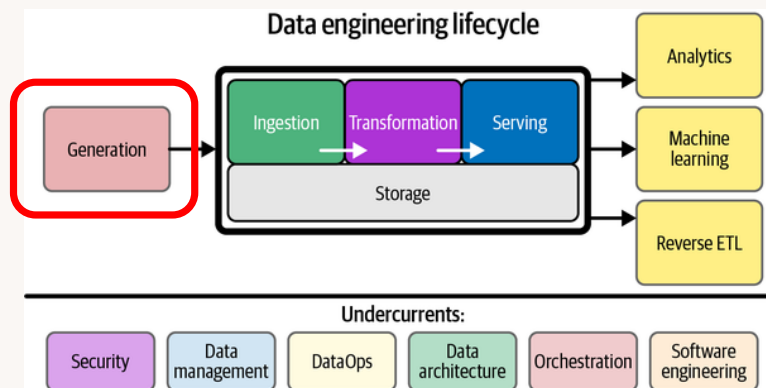
Analytics Solutions Bootcamp

14

Data engineering lifecycle

Data engineering lifecycle

>>



August 28, 2026

Analytics Solutions Bootcamp

15

How is data created?



Data is created by in two ways: **analog** and **digital**.

- Analog data creation occurs in the real world (vocal speech, sign language, writing on paper, playing an instrument).
- Digital data is created by converting analog data to digital form (mobile texting app, credit card transaction).

Types of data



Data can be categorized into three.

- **Structured**: Highly organized information that fits neatly into rows and columns with a fixed schema (spreadsheets, SQL databases)
- **Semi-structured**: Data that does not use a rigid table format, but contains tags, markers, or key-value pairs to show hierarchies (JSON, XML)
- **Unstructured**: Raw information with no predefined data model or organization (media files, PDF documents, social media posts)

Source systems



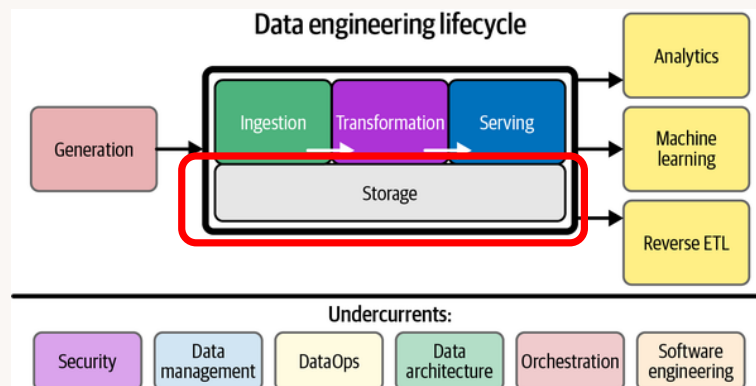
Here are the main source systems where data is created or sourced from.

1. Files and unstructured data
2. APIs
3. Application databases
4. Message and streams

Data engineering lifecycle

Data engineering lifecycle

>>



August 28, 2026

Analytics Solutions Bootcamp

19

OLTP and OLAP patterns



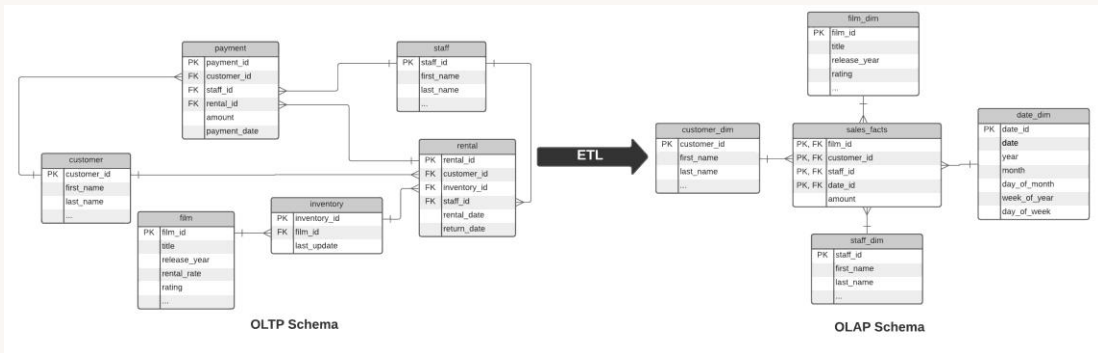
OLTP. OLTP, or Online Transaction Processing, is a pattern by which we access and manipulate data in the database transaction-by-transaction. A transaction refers to a single unit of work, such as a money transfer, a book, a blog post, and so on.

OLAP. Analytic queries consume a large number of records, look for only a few specific columns of each record, and often aggregate data to calculate statistics (min, max, sum, average,...). This pattern of accessing data in the database is called Online Analytic Processing (OLAP).

Data engineering lifecycle – Storage

OLTP and OLAP patterns

>>



August 28, 2026

Analytics Solutions Bootcamp

21

DE Storage Abstractions



Data engineering storage abstractions are data organization and query patterns that sit at the heart of the data engineering lifecycle and are built atop the data storage systems. Here are some considerations when choosing between these abstractions.

- Purpose and use case
- Update patterns
- Cost
- Separate storage and compute

Data engineering lifecycle – Storage

Data Warehouse



Data warehouses are a standard OLAP data architecture. It is an architecture for data centralization.

Data engineering lifecycle – Storage

Data Lake



Data lake was originally conceived as a massive data store where data was retained in raw, unprocessed form.

Data Lakehouse

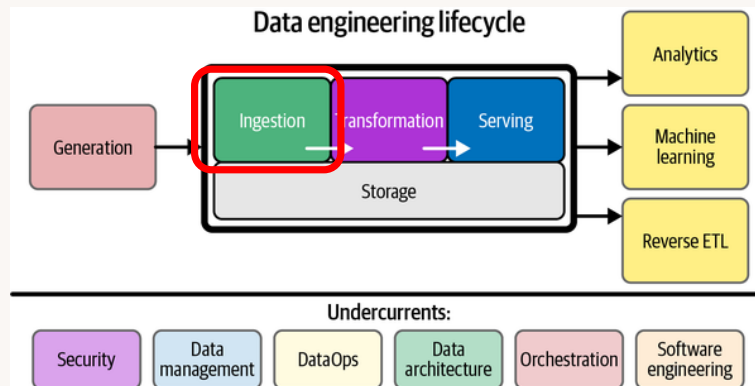


Data lakehouse combined the aspects of the data warehouse and the data lake. The lakehouse stored data in object storage (similar to data lake) but adds robust and schema support and features similar to a data warehouse.

Data engineering lifecycle

Data engineering lifecycle

>>



August 28, 2026

Analytics Solutions Bootcamp

26

What is data ingestion



Data ingestion is the process of moving data from one place to another.

A **data pipeline** is the combination of architecture, systems, and processes that move data through the stages of the data engineering lifecycle.

Ingestion frequency



How often do we ingest or “refresh” data?

- Batch (monthly, weekly, daily)
- Micro-batch (every 5 mins, every minute)
- Real-time (in seconds, “near real-time”)

“If nothing breaks when it’s 5 minutes late, you probably don’t need streaming.”

https://www.reddit.com/r/dataengineering/comments/1knjbpf/batch_vs_microbatch_vs_streaming_what_i_learned/

Schema and data types



Schema refers to the structure of the data. Typically, it includes what data elements are listed and their corresponding data types.

- Pipeline breaks when target schema does not match source schema.
- Schema changes are uncontrollable. It can be handled automatically or manually.
- Data lineage will be helpful in checking downstream impact.

Ways to ingest data



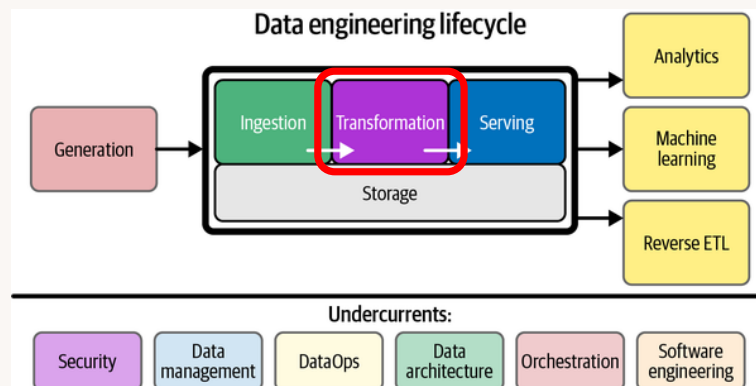
Here are the common ways to ingest data

1. Direct database connection.
2. Change data capture (CDC).
3. APIs
4. Message queues and event streams

Data engineering lifecycle

Data engineering lifecycle

>>



August 28, 2026

Analytics Solutions Bootcamp

31

Queries



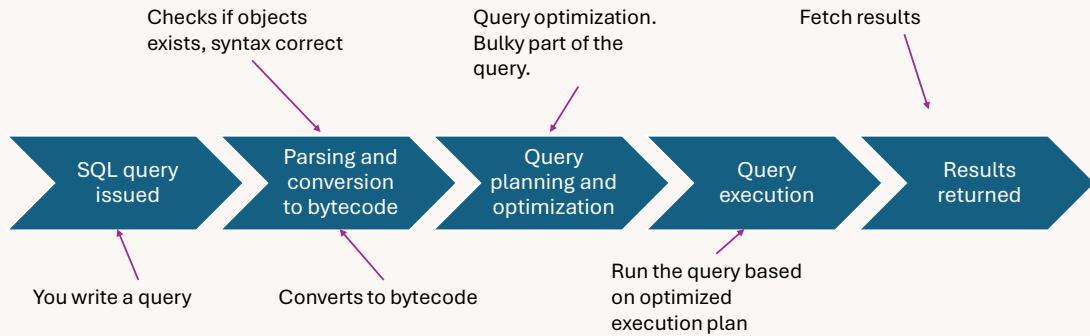
- A query allows you to receive and act on data.
- Query languages:
 - DQL: Data query language. SELECT
 - DDL: Data definition language. CREATE, DROP, ALTER
 - DML: Data manipulation language. INSERT, DELETE, UPDATE, MERGE
 - DCL: Data control language. GRANT, DENY, REVOKE
 - TCL: Transaction control language. COMMIT, ROLLBACK

Example of transaction



```
BEGIN TRANSACTION;  
DELETE FROM users  
WHERE id = 123;  
--Verify the deletion  
SELECT * FROM users WHERE id = 123;  
-- If everything looks correct:  
COMMIT;  
-- Otherwise:  
-- ROLLBACK;
```

Life of a query



Query execution order



- | | |
|----------------|-------------|
| 1. FROM + JOIN | 6. DISTINCT |
| 2. WHERE | 7. ORDER BY |
| 3. GROUP BY | 8. LIMIT |
| 4. HAVING | 9. OFFSET |
| 5. SELECT | |

Improving query performance

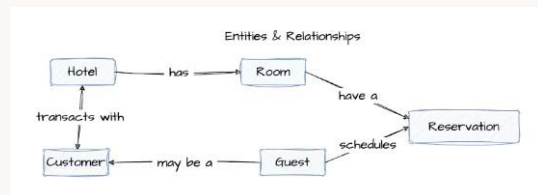


- Optimize join strategy and schema; understand granularity
- Use the explain plan and understand your queries performance
- Avoid full table scans
- Leverage the indices or partitions of the data

Data modeling concepts



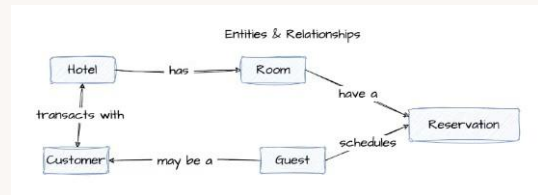
A data model represents the way data relates to the world. A data model abstracts and visualizes a certain process.



Conceptual data model



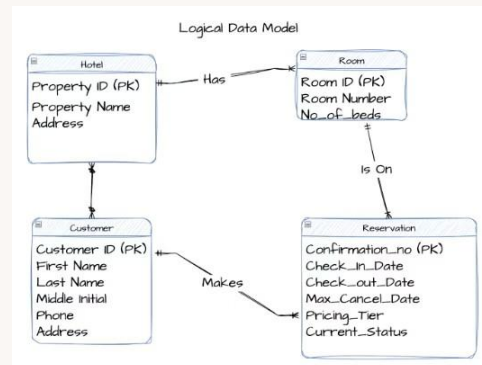
A conceptual data model is a high-level, visual blueprint that maps out core business entities and relationships.



Logical data model



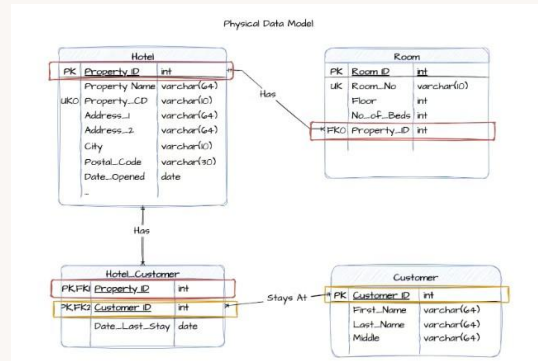
A logical data model is a structured representation of data that outlines entities, attributes, and relationships without referencing any specific database or technology.



Physical data model



A physical data model is a technical blueprint that maps your logical data model to a specific database system. It outlines exactly how data will be stored, indexed, and accessed, down to the datatype, column length, and constraints.



Normalization



Normalization is a database data modeling practice that enforces strict control over the relationships of tables and columns within a database.

- Remove data redundancy
- Ensure referential integrity

Normalization forms



Codd introduced normal forms.

1. **First normal form (1NF).** Each column is unique and has a single value. The table has a unique primary key.
2. **Second normal form (2NF).** The requirements of 1NF, plus partial dependencies are removed.
3. **Third normal form (3NF).** The requirements of 2NF, plus each table contains only relevant fields related to its primary key and has no transitive dependencies.

Denormalized



OrderDetail

OrderID	OrderItems	CustomerID	CustomerName	OrderDate
100	[{"sku": 1, "price": 50, "quantity": 1, "name": "Thingamajig"}, {"sku": 2, "price": 25, "quantity": 2, "name": "Whatchamacallit"}]	5	Joe Reis	2022-03-01

First normal form (1NF)



First normal form (1NF). Each column is unique and has a single value. The table has a unique primary key.

OrderDetail

OrderID	OrderItems	CustomerID	CustomerName	OrderDate
100	[{"sku": 1, "price": 50, "quantity": 1, "name": "Thingana jlg"}, {"sku": 2, "price": 25, "quantity": 2, "name": "Whatchanacallit"}]	5	Joe Reis	2022-03-01

OrderDetail with LineItemNumber column

Order ID	LineItem Number	Sku	Price	Quantity	Product Name	Customer ID	Customer Name	OrderDate
100	1	1	50	1	Thingana jlg	5	Joe Reis	2022-03-01
100	2	2	25	2	Whatchana callit	5	Joe Reis	2022-03-01
101	1	3	75	1	Whozee whatzit	7	Matt Housley	2022-03-01
102	1	1	50	1	Thingana jlg	7	Matt Housley	2022-03-01

Second normal form (2NF)



Second normal form (2NF). The requirements of 1NF, plus partial dependencies are removed.

OrderDetail with LineItemNumber column

Order ID	LineItem Number	SKU	Price	Quantity	Product Name	Customer ID	Customer Name	OrderDate
100	1	1	50	1	Thingana jig	5	Joe Reis	2022-03-01
100	2	2	25	2	Whatchana callit	5	Joe Reis	2022-03-01
101	1	3	75	1	Whozee whatzit	7	Matt Housley	2022-03-01
102	1	1	50	1	Thingana jig	7	Matt Housley	2022-03-01

Orders

OrderID	CustomerID	CustomerName	OrderDate
100	5	Joe Reis	2022-03-01
101	7	Matt Housley	2022-03-01
102	7	Matt Housley	2022-03-01

OrderLineItem

OrderID	LineItemNumber	SKU	Price	Quantity	ProductName
100	1	1	50	1	ThinganaJig
100	2	2	25	2	WhatchanaCallit
101	1	3	75	1	WhozeeWhatzit
102	1	1	50	1	ThinganaJig

Third normal form (3NF)



Third normal form (3NF). The requirements of 2NF, plus each table contains only relevant fields related to its primary key and has no transitive dependencies.

Orders			
OrderID	CustomerID	CustomerName	OrderDate
100	5	Joe Reits	2022-03-01
101	7	Matt Housley	2022-03-01
102	7	Matt Housley	2022-03-01

OrderLineItem					
OrderID	LineItemNumber	SKU	Price	Quantity	ProductName
100	1	1	50	1	Thinganajig
100	2	2	25	2	Whatchanacallit
101	1	3	75	1	Whozeewhatit
102	1	1	50	1	Thinganajig

OrderLineItem					
OrderID	LineItemNumber	SKU	Price	Quantity	
100	1	1	50	1	
100	2	2	25	2	
101	1	3	75	1	
102	1	1	50	1	

SKUs	
SKU	ProductName
1	Thinganajig
2	Whatchanacallit
3	Whozeewhatit

Dimensional modeling

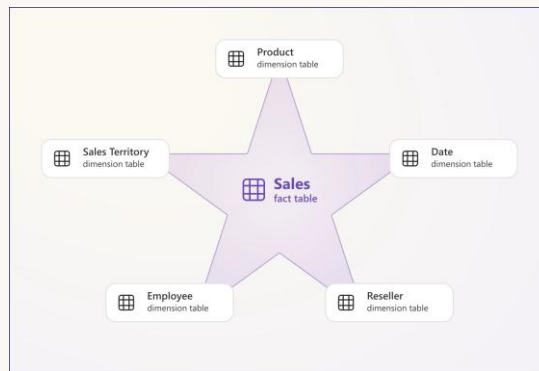


In dimensional data modeling, the world is divided into measurements (facts) and contexts (dimensions).

- Facts are numeric values
- Dimensions describe the WH-questions the facts
- Eg. Sales in pesos by branch, month, and product
- Understandability of the users and fast query performance
- Star schema for denormalized, snowflake for normalized

Data engineering lifecycle – Transformation

Dimensional model – star schema >>

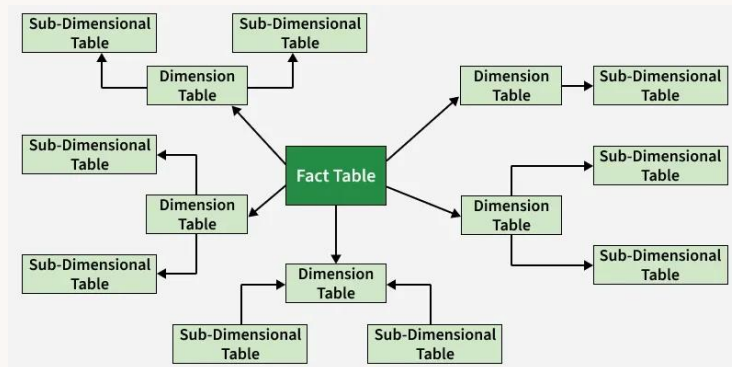


August 28, 2026

Analytics Solutions Bootcamp

48

Dimensional model – snowflake >>



Activity: Dim or Fact?



Item	Dim or Fact?
Sales	Fact
Date	Dim
Attendance	Fact
City	Dim
Promotion	Dim
Profit	Fact
Social media engagement	Fact
Discounts	Fact
Age	Dim
Quantity	Fact
Longitude / Latitude	Dim

Slowly changing dimension (SCD) >>

A slowly changing dimension tracks changes in dimensions.

- Type 0: no changes
- Type 1: Overwrite existing records
- Type 2: Keep a full history of dimension records.
- Type 3: Keep the last update and current value in separate columns.

SCD 1: Overwrite records



Source:

user_id=123 updates his email
from jsmith@gmail.com to
jsmith@outlook.com.

Old state	
user_id	email
123	jsmith@gmail.com

Current state	
user_id	email
123	jsmith@outlook.com

SCD 2: Keep history of changes >>

Source:

user_id=123 updates his zip code from 1702 to 1612. Then, updates the zip code again to 3006.

user_id	zip	start_date	end_date	current
123	1702	2024-03-01	NULL	Y

user_id	zip	start_date	end_date	current
123	1702	2024-03-01	2024-12-09	N
123	1612	2024-12-10	NULL	Y

user_id	zip	start_date	end_date	current
123	1702	2024-03-01	2024-12-09	N
123	1612	2024-12-10	2025-10-03	N
123	3006	2025-10-04	NULL	Y

SCD 3: Past and present values >>

Source:

user_id=123 updates his zip code from 1702 to 1612. Then, updates the zip code again to 3006.

user_id	current_zip	previous_zip
123	1702	NULL

user_id	current_zip	previous_zip
123	1612	1702

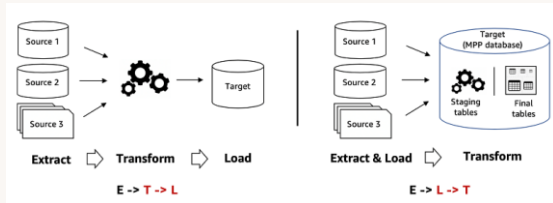
user_id	current_zip	previous_zip
123	3006	1612

Data engineering lifecycle – Transformation

ETL vs ELT



The main difference between ETL and ELT is when transformation happens. In ETL, transformation happens before loading to DW, while in ELT, raw data is loaded to DW before processing.



Update patterns

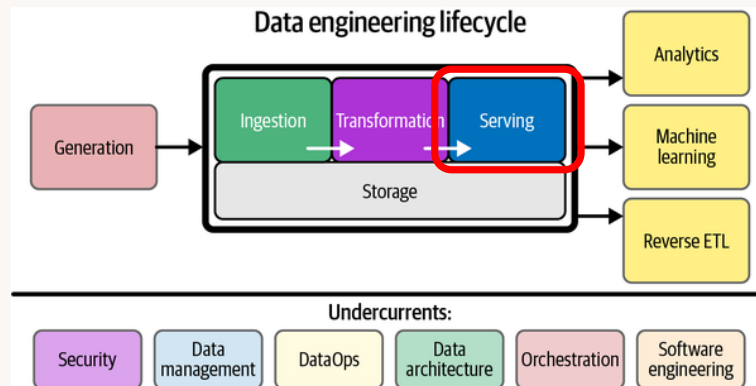


- Truncate and reload (full-load)
- Insert only (append)
- Delete-insert (DelSert)
- UpSert / MERGE
- Schema updates
- Data wrangling
- Business logic and derived data

Data engineering lifecycle

Data engineering lifecycle

>>



August 28, 2026

Analytics Solutions Bootcamp

57

Serving data and data products >>

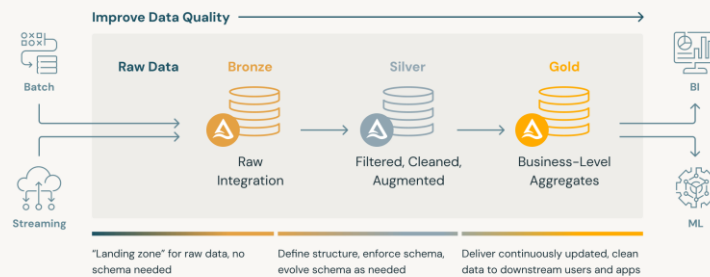
- Considerations when serving data: **trust**, **use cases**, and the **users**.
- Some data products are as follows:
 - Extracts, APIs, reports, dashboards
 - Operational and real-time analytics
 - AI and machine learning solutions

Data engineering lifecycle – Serving

Medallion Architecture



Building reliable, performant data pipelines with  DELTA LAKE



August 28, 2026

Analytics Solutions Bootcamp

59

Data engineering lifecycle – Serving

Semantic and Metrics Layer >>

A metrics layer is a tool for maintaining and computing business logic. Similarly, semantic layer bridges the gap between the technical structure of the underlying data

Serving data for data science >>

Data engineers play an important role in serving high-quality data for data science use cases.

- Processed data can already be shared with data scientists
- Train-validation-test split can already be made

Introduction to modern data stack

Introducing concepts to modern approach to solving data engineering problems

August 28, 2026

Analytics Solutions Bootcamp

62

On-premises vs cloud solutions >>

On-premises server is a physical computer that stores data and runs programs locally within an organization's own building.

Cloud means "someone else's computer or server". It is a virtual server hosted on a public or private cloud network.

Modern data stack

Legacy vs Modern Data Stack >>

- Infrastructure: LDS relies on physical servers; MDS is cloud native.
- Scalability: LDS requires manual scaling; MDS scales dynamically with demand.
- Integration: LDS relies on custom workflows; MDS automates data ingestion.

Modern data stack

Legacy vs Modern Data Stack >>

- Flexibility: LDS is monolithic; MDS is modular, enabling seamless tool integration.
- Analytics: LDS supports batch reporting; MDS offers real-time insights and interactive dashboards.
- Cost: LDS involves significant upfront investment; MDS uses pay-as-you-go models.

Components of MDS

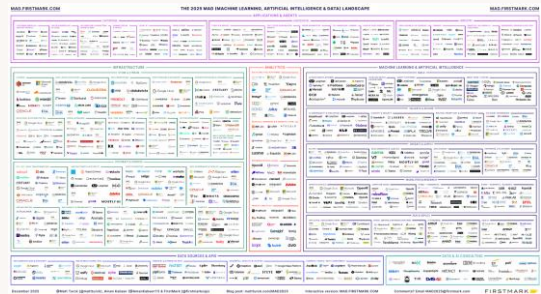
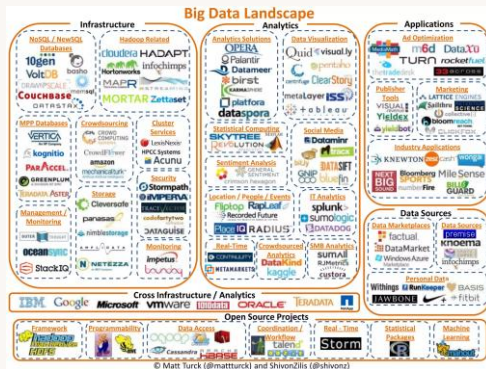


The MDS consists of several core components.

- Cloud native data storage, warehouses, or lakehouses
- Automated data ingestion and real-time processing
- Automated workflows for data transformations
- BI and analytics layer serving interactive and real-time dashboards
- Seamless data observability for data quality and reliability monitoring

Modern data stack

Big data landscape 2012 vs 2025 >>



August 28, 2026

Analytics Solutions Bootcamp

67

Modern data stack

Takeaways



Tools will evolve. Core skills will never go away!

- Be good with SQL, Python, shell, or JVM-based languages
- Learn hardware concepts, such as servers, networks, storage, RAM
- Data modeling skills are important
- Be good with optimizing scripts and queries

References

- Association, D. M., Henderson, D., & Earley, S. (2017b). DAMA-DMBOK: Data Management Body of Knowledge.
- Devlin, B. A., & Murphy, P. T. (1988). *An architecture for a business and information system*. *IBM Systems Journal*, 27(1), 60–80.
<http://altaplana.com/ibmsj2701G.pdf>
- GeeksforGeeks. (2025, July 15). *Difference between Kimball and Inmon*. <https://www.geeksforgeeks.org/dbms/difference-between-kimball-and-inmon/>
- Reis, J., & Housley, M. (2022). *Fundamentals of Data Engineering: Plan and Build Robust Data Systems*. O'Reilly Media.

References

- Williams, P. (2012, August 23). A short history of data warehousing. DATAVERSITY. <https://www.dataversity.net/articles/a-short-history-of-data-warehousing/ln>
- Rivera, S. (n.d.). *Conceptual vs. logical vs. physical data models: What's the difference?* ThoughtSpot. <https://www.thoughtspot.com/data-trends/data-modeling/conceptual-vs-logical-vs-physical-data-models>
- Jalao, E. R. L. (2024). Applied analytics for data analysts, engineers, and scientists. University of the Philippines Press

References

- Amazon Web Services. (n.d.). What's the difference between ETL and ELT? <https://aws.amazon.com/compare/the-difference-between-etl-and-elt/>
- Databricks. (2023, August 7). *What is Medallion architecture?* <https://www.databricks.com/blog/what-is-medallion-architecture>
- Krantz, T., & Jonker, A. (n.d.). *What is the modern data stack?* IBM Think. <https://www.ibm.com/think/topics/modern-data-stack>